Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory, Edge Cases & Code Examples

- ✓ Overview
- ✓ Lock-Free Programming Foundation
- ✓ Memory Ordering and Visibility
- ✓ Memory Ordering Visual Guide
- ✓ Real-Time Systems: Autonomous Driving

→ *Swipe to learn* →

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 1. std::atomic - Overview (1/2)

Definition:

- Template class introduced in C++11 for atomic operations on shared variables
- Operations execute as single, uninterruptible units
- Prevents race conditions without explicit mutex locks
- Defined in `<atomic>` header

Core Guarantee:

Aspect	std::atomic	Regular Variable
Atomicity	✅ All operations indivisible	❌ Multi-step operations can interleave
Visibility	✅ Configurable memory ordering	❌ No guarantees
Thread Safety	✅ Concurrent access safe (with proper ordering)	❌ Data races on concurrent access
Hardware Support	✅ Uses CPU atomic instructions	Uses normal load/store

How It Works:

```
std::atomic<int> counter(0);

// Non-atomic (UNSAFE):
// int temp = counter;  // Read
// temp = temp + 1;     // Modify
// counter = temp;      // Write  ← Another thread can interleave here!

// Atomic (SAFE):
counter.fetch_add(1);  // Single atomic operation (LOCK INC on x86, LDREX/STREX on ARM)
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 1. std::atomic - Overview (2/2)

Hardware-Level Operations:

Platform	Instruction	Purpose
x86/x64	LOCK CMPXCHG	Atomic compare-and-swap
x86/x64	LOCK INC/DEC	Atomic increment/decrement
ARM	LDREX/STREX	Load-exclusive / Store-exclusive
ARM	DMB	Data memory barrier (ordering)

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 2. Lock-Free Programming Foundation (1/2)

Definitions:

Concept	Guarantee	Example
Blocking	Threads wait for locks (can halt indefinitely)	std::mutex
Lock-Free	System as whole makes progress (individual threads may retry)	CAS-based stack
Wait-Free	Every thread completes in bounded steps	fetch_add counter

Lock-Free vs Wait-Free:

```
// Lock-Free (not wait-free):
void push(T value) {
    Node* new_node = new Node{value};
    do {
        new_node->next = head.load();
    } while (!head.compare_exchange_weak(new_node->next, new_node));
    // ↑ Threads may retry indefinitely (lock-free), but system progresses
}

// Wait-Free:
uint64_t increment() {
    return counter.fetch_add(1);  // Completes in bounded steps (no loop)
}
```

Lock-Free Building Block:

Operation	Purpose	Typical Use
Compare-And-Swap (CAS)	Atomic conditional update	Lock-free stack push/pop
fetch_add	Atomic increment/return old	Wait-free counters, indices
exchange	Atomic swap/return old	Lock-free ownership transfer

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 2. Lock-Free Programming Foundation (2/2)

Trade-offs:

Approach	Pros	Cons
Mutex-Based	Simple, protects complex critical sections	Context switches, priority inversion, potential deadlock
Lock-Free	No blocking, high contention performance	Complex (ABA problem), requires memory ordering expertise
Wait-Free	Bounded latency per thread	Limited operations (mostly wait-free only for simple ops)

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 3. Memory Ordering and Visibility (1/2)

The Problem: Reordering

Modern CPUs and compilers reorder instructions for performance:

```
int data = 0;
bool ready = false;

// Writer:
data = 42;      // Instruction 1
ready = true;   // Instruction 2

// Compiler/CPU may reorder to:
ready = true;   // Executed first!
data = 42;      // Executed second

// Reader:
if (ready) {
    use(data);  // ❌ Might see data = 0 (reordering!)
}
```

The Five Memory Orderings:

Ordering	Atomicity	Ordering Constraints	Cost	Use Case
relaxed	✅	❌ None (maximum reordering)	Fastest	Independent counters/stats
acquire	✅	✅ No read/write reorders before this load	Fast	Loading flags
release	✅	✅ No read/write reorders after this store	Fast	Storing flags
acq_rel	✅	✅ Both acquire and release	Moderate	Read-modify-write (RMW)
seq_cst	✅	✅ Global total order across all threads	Slowest	Complex multi-variable logic

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 3. Memory Ordering and Visibility (2/2)

Acquire-Release Example:

```
std::atomic<bool> ready(false);
int data = 0;

// Writer thread:
data = 42;  // Happens-before the release
ready.store(true, std::memory_order_release);  // Release: all prior writes visible

// Reader thread:
while (!ready.load(std::memory_order_acquire)) {}  // Acquire: see all prior writes
assert(data == 42);  // ✅ Guaranteed to see data = 42
```

Happens-Before Relationship:

Relationship	Meaning
Program Order	Within a thread, statements execute in order (as written)
Synchronizes-With	Release store ↔ Acquire load on same atomic creates edge
Happens-Before	Transitive closure: A happens-before B → A's effects visible to B

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 4. Memory Ordering Visual Guide

Sequential Consistency (seq_cst) - Global Total Order:

```
Thread 1: [x.store(1)] ──→ [y.store(1)]
                              ↓
Thread 2:                [y.load() sees 1] ──→ [x.load() sees 1]

All threads observe same order: x=1 before y=1
```

Acquire-Release - Pairwise Sync:

```
Thread 1: [data=42] ──→ [ready.store(true, release)]
                              ↓ synchronizes-with
Thread 2:           [ready.load(true, acquire)] ──→ [see data=42]

Only threads with acquire-release pair synchronize
```

Relaxed - No Ordering:

```
Thread 1: [data=42] ──→ [ready.store(true, relaxed)]
          ↓ may reorder!     ↓
Thread 2: [ready.load(relaxed) sees true]
          ↓ may NOT see data=42 (no ordering guarantee)
```

📌 5. Real-Time Systems: Autonomous Driving (1/2)

Sensor Fusion Pipeline Requirements:

Component	Frequency	Latency Requirement	Challenge
LiDAR Processing	10-20 Hz	< 50ms per frame	1M+ points/frame
Camera Capture	30 Hz	< 5ms wake-up	Multiple cameras (6-8)
Radar Processing	20 Hz	< 30ms	Concurrent sensor streams
Sensor Fusion	30 Hz	< 10ms total pipeline	Timestamp synchronization

Why Atomics Are Critical:

Operation	Mutex Approach	Atomic Approach
Frame Counter Update	Lock, increment, unlock (~1-10μs + context switch risk)	fetch_add (~50ns, no syscall)
Ready Flag Check	Lock, check, unlock (~1-10μs)	load (~10ns)
Timestamp Capture	Lock, read clock, store, unlock	exchange with timestamp (~100ns)
Point Cloud Queue	Blocking queue with mutex (potential priority inversion)	Lock-free queue (bounded latency)

📌 5. Real-Time Systems: Autonomous Driving (2/2)

Example: Multi-Camera Frame Counting:

```
// ❌ Mutex approach (slow):
std::mutex mtx;
uint64_t frame_id = 0;

void on_camera_frame(int camera_id) {
    std::lock_guard<std::mutex> lock(mtx);  // ~1-10µs overhead
    uint64_t id = frame_id++;
    process_frame(camera_id, id);
}

// ✅ Atomic approach (fast):
std::atomic<uint64_t> frame_id{0};

void on_camera_frame(int camera_id) {
    uint64_t id = frame_id.fetch_add(1, std::memory_order_relaxed);  // ~50ns
    process_frame(camera_id, id);
}
```

Performance Impact:

Metric	Mutex	Atomic (relaxed)	Speedup
Operation latency	1-10µs	50ns	20-200x
Context switch risk	High (syscall)	None (user-space)	N/A
Priority inversion risk	Yes	No	Critical for RT
Throughput (ops/sec)	~100K	~20M	200x

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 6. Atomic Operations Quick Reference

Basic Operations:

```
std::atomic<int> x(10);

// Loads and Stores:
int val = x.load(std::memory_order_acquire);
x.store(20, std::memory_order_release);

// Read-Modify-Write (RMW):
int old = x.exchange(30);           // Swap
int prev = x.fetch_add(5);          // Add, return old
int prev2 = x.fetch_sub(2);         // Subtract, return old

// Compare-And-Swap (CAS):
int expected = 10;
bool success = x.compare_exchange_strong(expected, 20);
// If x==10: x=20, return true
// If x!=10: expected=current_x, return false
```

Operation Complexity:

Operation	Complexity	Hardware
load	O(1)	Simple load with barrier
store	O(1)	Simple store with barrier
fetch_add/sub	O(1)	LOCK ADD (x86) or LDREX/STREX loop (ARM)
exchange	O(1)	XCHG (x86) or LDREX/STREX (ARM)
compare_exchange	O(1)	CMPXCHG (x86) or LDREX/STREX (ARM)

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Theory

📌 7. Common Pitfalls and Best Practices

Pitfalls:

Mistake	Problem	Fix
Using `volatile` for threading	No atomicity or ordering	Use `std::atomic`
Mixing atomic/non-atomic access	Data race → undefined behavior	All access must be atomic
CAS without loop	Fails under contention or spuriously	`while (!CAS) {}`
Relaxed without analysis	Reordering breaks assumptions	Start with seq_cst
Immediate deletion in lock-free	ABA problem, use-after-free	Hazard pointers / epoch-based reclamation
False sharing	Cache line bouncing → 10-100x slowdown	`alignas(64)` or padding

Best Practices:

- ✅ Start with `memory_order_seq_cst`, optimize to acquire-release only in proven hotspots
- ✅ Always use CAS in loops (especially `compare_exchange_weak`)
- ✅ Check `is_lock_free()` on target platform for performance-critical atomics
- ✅ Use `alignas(64)` for frequently updated independent atomics (avoid false sharing)
- ✅ Use acquire-release for producer-consumer flags/pointers
- ✅ Use relaxed only for truly independent counters/statistics
- ❌ Never use `volatile` for multithreading
- ❌ Never mix atomic and non-atomic access to same variable

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Edge Cases

📌 1. The ABA Problem in Lock-Free Structures

```
struct Node {
    int value;
    Node* next;
};

std::atomic<Node*> head(nullptr);

// Thread T1 reads head = A
Node* old_head = head.load();  // head points to node A

// Thread T2 pops A, pops B, pushes A back (same address!)
// Now head = A again (same pointer value)

// Thread T1's CAS succeeds even though list structure changed!
bool success = head.compare_exchange_strong(old_head, old_head->next);  // ❌ ABA!
```

The ABA problem occurs when a value changes from A to B and back to A, causing compare-and-swap to succeed even though the data structure's state has fundamentally changed. The pointer value matches, but the memory may have been freed and reallocated. This leads to use-after-free bugs and data corruption in lock-free stacks and queues.

Solutions include tagged pointers where a version counter is packed with the pointer, hazard pointers that delay deletion until no thread references the node, or epoch-based reclamation used in Linux RCU where nodes are freed only after all threads have passed through a grace period.

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Edge Cases

📌 2. Spurious Failures in compare_exchange_weak

```
std::atomic<int> counter(0);

void increment() {
    int expected = counter.load();
    // ❌ WRONG: weak CAS without loop
    if (counter.compare_exchange_weak(expected, expected + 1)) {
        std::cout << "Success\n";
    } else {
        std::cout << "Failed\n";  // May fail even if counter == expected!
    }
}

// ✅ CORRECT: weak CAS in loop
void increment_correct() {
    int expected = counter.load();
    while (!counter.compare_exchange_weak(expected, expected + 1)) {
        // expected updated with current value on failure
    }
}
```

compare_exchange_weak is allowed to fail spuriously even when the expected value matches the atomic value. This optimization enables faster code generation on architectures like ARM where load-linked/store-conditional instructions may fail due to cache line contention or context switches. Always use weak CAS in loops. Use compare_exchange_strong only when retries are prohibitively expensive.

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Edge Cases

📌 3. Memory Reordering Without Acquire-Release

```
int data = 0;
std::atomic<bool> ready(false);

// Writer thread
void writer() {
    data = 42;  // Store to data
    ready.store(true, std::memory_order_relaxed);  // ❌ Relaxed ordering
}

// Reader thread
void reader() {
    while (!ready.load(std::memory_order_relaxed)) {}  // ❌ Relaxed ordering
    assert(data == 42);  // May FAIL! data write might not be visible
}
```

With memory_order_relaxed, the compiler and CPU can reorder the write to data after the write to ready, or the reader may see ready as true but data as 0 due to cache coherence delays. Relaxed atomics guarantee atomicity but no cross-thread ordering.

The fix requires memory_order_release on the store and memory_order_acquire on the load, establishing a happens-before relationship where all writes before the release are visible to the acquirer.

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Edge Cases

📌 4. Use-After-Free in Lock-Free Pop

```
Node* pop() {
    Node* old_head = head.load();
    while (old_head && !head.compare_exchange_weak(old_head, old_head->next));

    if (old_head) {
        int val = old_head->value;
        delete old_head;  // ❌ Another thread may still be accessing old_head!
        return val;
    }
    return -1;
}
```

After successfully popping a node, another thread may still be in the compare-and-swap loop with a reference to the same node. Deleting it immediately causes undefined behavior. Safe memory reclamation strategies include hazard pointers (thread announces which nodes it accesses), epoch-based reclamation (nodes freed after global epoch advances), or reference counting with atomic shared_ptr.

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Edge Cases

📌 5. False Sharing with Atomic Counters

```
struct Counters {
    std::atomic<int> c1;  // Likely on same cache line (64 bytes)
    std::atomic<int> c2;
    std::atomic<int> c3;
    std::atomic<int> c4;
};

Counters counters;

void thread1() { counters.c1.fetch_add(1); }
void thread2() { counters.c2.fetch_add(1); }  // Cache line bounces between cores!
```

Even though c1 and c2 are independent atomics, if they reside on the same cache line, modifications cause cache line invalidation across cores, creating severe performance degradation. Solution: pad atomics to separate cache lines using alignas(64) or std::hardware_destructive_interference_size in C++17.

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 1. Basic Atomic Counter

This demonstrates atomic counter operations using fetch_add which atomically increments and returns the previous value. The operation uses memory_order_seq_cst by default, ensuring strong ordering. For performance-critical code where ordering is not needed, use fetch_add with memory_order_relaxed.

```
#include <atomic>
#include <thread>
#include <vector>
#include <iostream>

std::atomic<int> counter(0);

void increment_worker() {
    for (int i = 0; i < 100000; ++i) {
        counter.fetch_add(1);  // ✅ Atomic increment
    }
}

int main() {
    std::vector<std::thread> threads;
    for (int i = 0; i < 10; ++i) {
        threads.emplace_back(increment_worker);
    }

    for (auto& t : threads) {
        t.join();
    }

    std::cout << "Final counter: " << counter.load() << "\n";  // Always 1000000
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 2. Acquire-Release Synchronization

Acquire-release semantics establish a synchronizes-with relationship between threads. The release store ensures all preceding memory operations complete before the atomic write becomes visible. The acquire load ensures all subsequent operations see the effects of operations before the release. This pattern is fundamental for signaling data readiness in lock-free pipelines.

```
#include <atomic>
#include <thread>
#include <cassert>

std::atomic<bool> ready(false);
int shared_data = 0;

void writer() {
    shared_data = 42;  // Non-atomic write
    ready.store(true, std::memory_order_release);  // ✅ Release: all previous writes visible
}

void reader() {
    while (!ready.load(std::memory_order_acquire)) {}  // ✅ Acquire: see all prior writes
    assert(shared_data == 42);  // Guaranteed to pass
}

int main() {
    std::thread t1(writer);
    std::thread t2(reader);
    t1.join();
    t2.join();
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 3. Lock-Free Stack with Compare-And-Swap (1/2)

This lock-free stack uses compare-and-swap to atomically update the head pointer. The CAS operation checks if head still equals the expected value and updates it atomically. If another thread modified head concurrently, CAS fails and the loop retries with the updated value. Note the ABA problem and use-after-free issues require production code to use hazard pointers or epoch-based reclamation.

```
#include <atomic>
#include <iostream>

template<typename T>
struct Node {
    T value;
    Node* next;
    Node(T val) : value(val), next(nullptr) {}
};

template<typename T>
class LockFreeStack {
private:
    std::atomic<Node<T>*> head;

public:
    LockFreeStack() : head(nullptr) {}

    void push(T value) {
        Node<T>* new_node = new Node<T>(value);
        new_node->next = head.load();  // Load current head

        // CAS loop: retry until successful
        while (!head.compare_exchange_weak(new_node->next, new_node)) {
            // On failure, new_node->next is updated with current head
        }
    }

    bool pop(T& result) {
        Node<T>* old_head = head.load();

        while (old_head && !head.compare_exchange_weak(old_head, old_head->next)) {
            // On failure, old_head updated with current head, retry
        }
// ...continues on next slide
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 3. Lock-Free Stack with Compare-And-Swap (2/2)

[Code continues across slides]

```
// ...continued from previous slide

        if (old_head) {
            result = old_head->value;
            delete old_head;  // ⚠️ Unsafe: see memory reclamation edge case
            return true;
        }
        return false;
    }
};

int main() {
    LockFreeStack<int> stack;
    stack.push(10);
    stack.push(20);
    stack.push(30);

    int val;
    while (stack.pop(val)) {
        std::cout << val << "\n";
    }
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 4. Relaxed Ordering for Independent Counters

For independent statistics counters where precise ordering is not required, memory_order_relaxed offers maximum performance. Relaxed operations guarantee atomicity but allow maximum reordering. This is suitable for counters where eventual consistency is acceptable, such as request counts in autonomous vehicle telemetry systems.

```
#include <atomic>
#include <thread>
#include <iostream>

class Statistics {
private:
    std::atomic<uint64_t> requests{0};
    std::atomic<uint64_t> errors{0};
    std::atomic<uint64_t> bytes_sent{0};

public:
    void record_request() {
        requests.fetch_add(1, std::memory_order_relaxed);  // ✅ No ordering needed
    }

    void record_error() {
        errors.fetch_add(1, std::memory_order_relaxed);
    }

    void record_bytes(uint64_t bytes) {
        bytes_sent.fetch_add(bytes, std::memory_order_relaxed);
    }

    void print_stats() {
        std::cout << "Requests: " << requests.load(std::memory_order_relaxed) << "\n";
        std::cout << "Errors: " << errors.load(std::memory_order_relaxed) << "\n";
        std::cout << "Bytes: " << bytes_sent.load(std::memory_order_relaxed) << "\n";
    }
};
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 5. Sequential Consistency for Complex Coordination

Sequential consistency provides the strongest memory ordering guarantee—a global total order of all atomic operations across all threads. While easiest to reason about, it is the most expensive. Use seq-cst when correctness is paramount and the algorithm's logic depends on observing operations in a specific order across multiple variables.

```
#include <atomic>
#include <thread>
#include <cassert>

std::atomic<bool> x{false};
std::atomic<bool> y{false};
std::atomic<int> z{0};

void write_x_then_y() {
    x.store(true, std::memory_order_seq_cst);  // ✅ Seq-cst: total order
    y.store(true, std::memory_order_seq_cst);
}

void read_y_then_x() {
    while (!y.load(std::memory_order_seq_cst)) {}
    if (x.load(std::memory_order_seq_cst)) {
        ++z;  // Always executes: x must be true
    }
}

int main() {
    std::thread t1(write_x_then_y);
    std::thread t2(read_y_then_x);
    t1.join();
    t2.join();
    assert(z.load() == 1);  // Guaranteed to pass
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 6. Atomic Flag for Spinlock (1/2)

std::atomic_flag provides the most basic atomic primitive guaranteed to be lock-free. The test_and_set operation atomically sets the flag and returns its previous value. This spinlock is suitable for very short critical sections in scenarios where the cost of kernel-level mutex operations is prohibitive, such as updating shared sensor fusion timestamps in autonomous driving stacks.

```
#include <atomic>
#include <thread>
#include <iostream>

class Spinlock {
private:
    std::atomic_flag flag = ATOMIC_FLAG_INIT;

public:
    void lock() {
        while (flag.test_and_set(std::memory_order_acquire)) {
            // Spin-wait: repeatedly check until flag is clear
        }
    }

    void unlock() {
        flag.clear(std::memory_order_release);
    }
};

Spinlock spinlock;
int shared_counter = 0;

void worker() {
    for (int i = 0; i < 100000; ++i) {
        spinlock.lock();
        ++shared_counter;  // Critical section
        spinlock.unlock();
    }
}

int main() {
    std::thread t1(worker);
    std::thread t2(worker);
// ...continues on next slide
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 6. Atomic Flag for Spinlock (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    t1.join();
    t2.join();
    std::cout << "Counter: " << shared_counter << "\n";  // Always 200000
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 7. Memory Fence for Ordering Non-Atomic Operations

Memory fences (barriers) synchronize memory without requiring atomic operations on specific variables. A release fence prevents preceding operations from reordering after subsequent relaxed stores. An acquire fence prevents subsequent operations from reordering before preceding relaxed loads. Fences enable using relaxed atomics with explicit ordering control, useful when optimizing hot paths in perception pipelines.

```
#include <atomic>
#include <thread>
#include <cassert>

int data1 = 0;
int data2 = 0;
std::atomic<bool> ready{false};

void writer() {
    data1 = 100;
    data2 = 200;
    std::atomic_thread_fence(std::memory_order_release);  // ✅ Fence: order all previous writes
    ready.store(true, std::memory_order_relaxed);  // Can use relaxed with fence
}

void reader() {
    while (!ready.load(std::memory_order_relaxed)) {}
    std::atomic_thread_fence(std::memory_order_acquire);  // ✅ Fence: see all prior writes
    assert(data1 == 100 && data2 == 200);  // Guaranteed to pass
}

int main() {
    std::thread t1(writer);
    std::thread t2(reader);
    t1.join();
    t2.join();
    return 0;
}
```

Atomics and Memory Ordering - Lock-Free Programming Fundamentals

Code Examples

📌 8. Wait-Free Counter with fetch_add (1/2)

This wait-free counter uses fetch_add which completes in a bounded number of steps regardless of other threads. Every thread makes progress without spinning or blocking. This pattern is ideal for autonomous vehicle sensor fusion where camera, lidar, and radar callbacks must record frame numbers without interference, ensuring real-time guarantees for perception-to-planning latency budgets.

```
#include <atomic>
#include <thread>
#include <vector>
#include <chrono>
#include <iostream>

std::atomic<uint64_t> frame_counter{0};

void sensor_callback(int sensor_id) {
    // Simulate sensor processing
    std::this_thread::sleep_for(std::chrono::milliseconds(10));

    uint64_t frame_num = frame_counter.fetch_add(1, std::memory_order_relaxed);
    std::cout << "Sensor " << sensor_id << " processed frame " << frame_num << "\n";
}

int main() {
    std::vector<std::thread> sensors;

    // Simulate 5 camera sensors processing frames concurrently
    for (int i = 0; i < 5; ++i) {
        sensors.emplace_back([i]() {
            for (int frame = 0; frame < 10; ++frame) {
                sensor_callback(i);
            }
        });
    }

    for (auto& t : sensors) {
        t.join();
    }

    std::cout << "Total frames: " << frame_counter.load() << "\n";
// ...continues on next slide
```

📌 8. Wait-Free Counter with fetch_add (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    return 0;
}
```

What You Learned

Key Takeaways

- ✓ Overview
- ✓ Lock-Free Programming Foundation
- ✓ Memory Ordering and Visibility
- ✓ Memory Ordering Visual Guide

→ Swipe to learn →